Project Report - Group 5

Project Title: Save My Semester

Group members:

Maleeka Ramadan - 900251407 - maleeka@aucegypt.edu

Habiba Shouman - 900251864 - habibashouman2006@aucegypt.edu

Salma Mohib - 900253273 - salmamohib@aucegypt.edu

Github Link: https://github.com/habibashouman2006-eng/Save_My_Semester.git

Milestone 1 **Milestone 2** **Milestone 3**

Project Idea and Motivation

Project Idea: **DESKTOP APP**

- An academic decision support system that helps students prioritize, recover from overload, and manage deadlines realistically.

Motivation:

- Last semester was extremely overwhelming, and the deadlines were all overlapping, and prioritization issues led to panic and procrastination. As a result, we decided to create an app that does the organisation and prioritization aspects for you based on many factors.

Problem Description

Academic stress and poor workload management are among the most common challenges faced by university students. Apps that currently exist, like calendars, often don't provide guidance on how to approach high workloads; they just tell you when something is due. Students often lack a system that helps them reason about their tasks, leading to them facing problems with time

management, being unable to prioritize tasks, and spending more time on low-weighted tasks than high-weight ones.

System Design + Structure

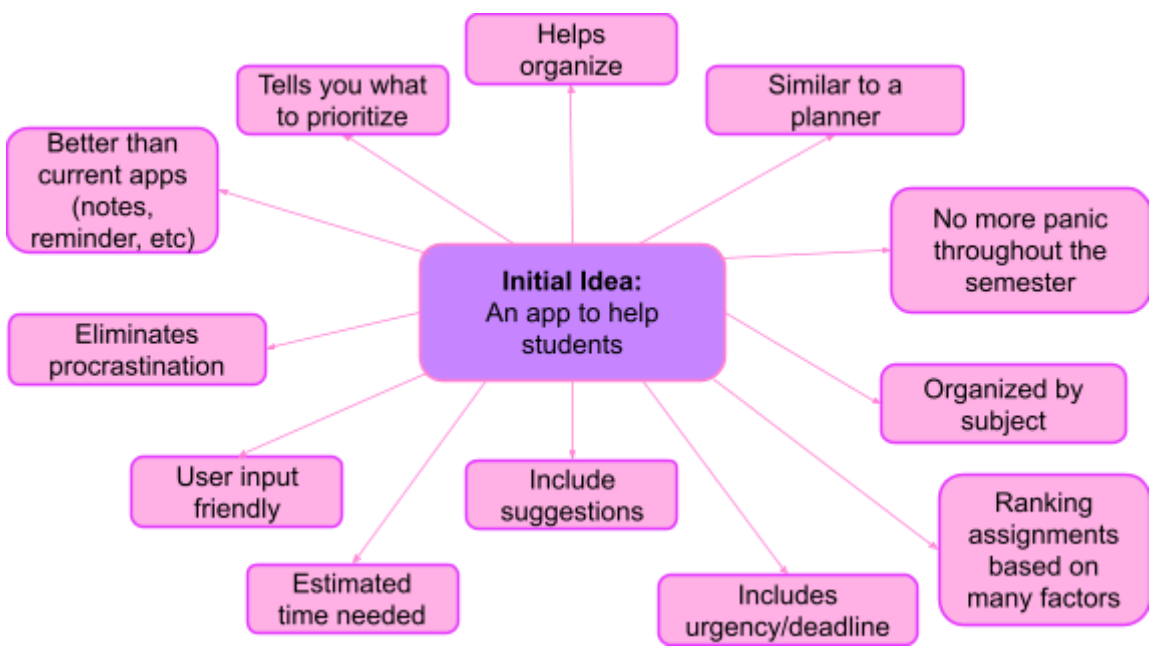
- The user would input the features of the new assignment they have to complete, which are mentioned on a later page, in order for the program of the application to analyze it by multiplying each task by a certain factor and determine its appropriate time of execution in the calendar based on its importance.
- The calendar in the application will consist of a continuous schedule in the form of a table, where there will be a feature for adding an assignment with its corresponding date of execution.
- The system is built around a single central data structure, the `Assignment` struct, which stores all task attributes including the computed priority score, urgency, time pressure, delay count, and completion status. This design keeps all task-related data in one place and makes it straightforward to pass tasks between functions.
- The application follows a modular architecture where each feature is implemented as a standalone function. All functions operate on a shared global vector of `Assignment` objects, which acts as the in-memory database for the session. This approach makes it easy to add or modify features independently without affecting unrelated parts of the system.
- The priority computation logic is centralized in a single `computePriority()` function that is called whenever a task is added or updated. A `recomputeAll()`

wrapper refreshes all scores before any display or analysis feature runs, ensuring the data is always current.

- Date arithmetic is handled without any external libraries. A custom day-counting function converts calendar dates to an absolute day number, allowing the system to calculate the number of days remaining until any deadline purely through integer arithmetic.
- The user interface is menu-driven, running in a continuous loop until the user chooses to exit. Input is handled with validation wrappers (`safeIntInput` and `safeDoubleInput`) that re-prompt the user on invalid entries rather than crashing or producing undefined behaviour.

Implementation Details + Progress

Brainstorming Ideas and Features:



<u>Final Features:</u>

1) **Inputs Assignment**

2) **Organizes by Subject**

3) **Deadlines**

4) **Estimated Duration of tasks**

5) **Difficulty of tasks**

6) **Priority / Grade weight**

7) **Assignment Progress**

8) **Calendar Form/Design**

9) **Overload Detection**

    - The app checks whether the total workload for a day or week is realistic.

10) **What-If Simulator**

    - Then the app shows how priorities or risk levels change.

      "What if I skip this task today?"

      "What if I only have 2 hours tonight?"

      "What if I finish this one first?"

11) **Task Clock + Break Timer**

    - Starts a clock when the user clicks "Start Task" & counts down based on the estimated time.
    - Suggests when to take breaks based on workload intensity.

12) **Screen Lock while tasks are in progress**

    - Like in exams, the user's screen is locked while completing the app to prevent distractions.

13) **Burnout / Stress reader → auto reschedule**

14) **Procrastination Pattern Tracker**

- Detects if the user keeps delaying long or difficult tasks.

15) **Small Wins Mode**

- Recommends quick tasks first when the user feels overwhelmed. If the priority score calculation is less than (a small predetermined number), then the program suggests it first, unless time does not allow it.

Since Milestone 1, the team has successfully implemented the core system in C++. The following features are now functional:

The assignment input system accepts all planned parameters from the user, assignment name, subject, deadline, estimated duration, difficulty level (1–5), grade weight (as a percentage of the final grade), and current progress percentage. Input validation is applied throughout to ensure the program handles incorrect entries gracefully.

The priority scoring algorithm has been fully implemented using the formula defined during planning: (Urgency × 4) + (Grade Weight × 3) + (Difficulty × 2) + (Time Pressure × 2) − (Progress × 2). Each factor is normalised to a 0–10 scale before being weighted, ensuring the score is consistent regardless of the raw input values. Urgency is derived from the number of days until the deadline, and time pressure is calculated by comparing the estimated task duration against the hours realistically available before the deadline.

The ranked task calendar displays all active assignments sorted by priority score in a formatted table, including deadline, difficulty, grade weight, progress, and an urgency label (Critical / High / Medium / Low).

The overload detection feature calculates the total remaining work hours due within the current week and compares them against a weekly capacity threshold. It also checks whether the tasks due today exceed a single day's available hours, and provides actionable suggestions when overload is detected.

The What-If Simulator allows the user to explore three scenarios: skipping a task and seeing how its urgency and priority shift; entering a limited number of available hours to see which tasks can realistically be completed; and choosing to finish one task first to see how the remaining task list reorganises.

The Task Clock implements a Pomodoro-based work and break timer. When a user starts a task, the system counts down a 25-minute work session followed by a 5-minute break, then automatically updates the task's progress percentage.

The Procrastination Pattern Tracker records how many times a task has been delayed or rescheduled. When a task has been delayed two or more times, the system flags it with a warning and suggests the user break it into smaller steps.

Small Wins Mode filters tasks with a low priority score and sorts them by shortest remaining duration, giving overwhelmed students a starting point that builds momentum.

The auto-reschedule feature detects overdue tasks and moves their deadline to the current day, incrementing their delay counter and recalculating their priority score.

The Burnout and Stress Estimator aggregates all active priority scores and counts the number of critically urgent tasks to produce a stress index, then outputs a risk level and personalised suggestions.

Challenges Encountered

- One of the first challenges was implementing date arithmetic in C++ without relying on external libraries.
- Calculating the number of days between today and a future deadline required writing a custom function to convert calendar dates into an absolute day count, accounting for leap years and varying month lengths. This was solved by building a `dateToDayNumber()` function that sums up all days from year 1 to the given date.
- Calibrating the priority score formula was another challenge. During initial testing, the weights produced scores that did not reflect real-world urgency in an intuitive way, for example, a low-weight assignment due tomorrow was ranking below a high-weight assignment due in three weeks. This was resolved by normalising all inputs to a 0–10 scale before applying the weights, and by adjusting the urgency multiplier to be the highest-weighted factor in the formula.
- Managing user input robustly took more effort than expected. C++ input streams can enter a fail state when the user types unexpected characters, which caused early versions of the program to loop infinitely or skip prompts. This was addressed by writing dedicated input validation functions that clear the stream and re-prompt the user whenever an invalid entry is detected.
- Designing the What-If Simulator required careful thought about how to represent hypothetical states without permanently modifying the task data. This was solved by working with local copies or pointer-based reads during simulation, so the original task list remains unchanged unless the user explicitly makes an update.

Solutions and Design Decisions (Critical Thinking)

Critical Thinking About AI-Generated Solutions:

While AI tools significantly accelerated development, the team applied critical thinking at several points.

- The initial priority formula suggested by AI used raw input values without normalisation, which caused the grade weight (on a 0–100 scale) to dominate the score and make urgency nearly irrelevant. The team identified this imbalance during testing and redesigned the normalisation step to bring all factors onto a 0–10 scale before weighting.
- The date calculation code generated by AI initially used the `<ctime>` library's `tm` struct directly for date arithmetic, which produced incorrect results around month boundaries. The team decided to replace this with a fully custom day-counting function that is easier to reason about and verify manually.
- In the What-If Simulator, the AI's first approach modified the actual task objects in the vector to simulate scenarios, which would have permanently altered the data. The team recognised this as a design flaw and revised the logic to use local copies during simulation.
- These experiences reinforced an important lesson: AI tools are effective at generating working first drafts quickly, but the team's own understanding of the problem is essential for catching subtle logical errors that the AI does not flag.

Explanation of how AI tools were Used During the Project

Planned AI Usage

- **ChatGPT/Gemini** → brainstorming, planning, formula refinement, report support
- **Claude** → coding help, debugging, implementation logic
- **GitHub Copilot** → in-editor code suggestions and faster coding

During Milestone 2, the team used AI tools in the following ways:

- **ChatGPT and Gemini** were used for brainstorming how to structure the priority formula and for researching different scheduling algorithms. They were also used to help draft and refine sections of the project report.
- **Claude** was used as the primary coding assistant. The team provided Claude with the full list of planned features, the priority score formula, and the project constraints (C++ only, no external libraries, console-based interface). Claude generated the initial implementation, which the team then compiled, tested, and reviewed together. Specific prompts focused on the date arithmetic problem, the input validation system, and the What-If Simulator logic.
- **GitHub Copilot** was used within the code editor to speed up repetitive coding tasks such as writing similar output formatting blocks and switch-case structures.

All AI-generated code was reviewed by the team before being accepted. The team compiled and ran the program after each major addition to verify correct behaviour, and made manual adjustments where the output did not match expectations.

Demo Video Links:

Milestone 2: …

Milestone 3: …

Individual Contributions of Each Team Member

All of us helped each other in their sections, but this is how we split the work:

Maleeka:

Report:	Slides:
Logo, Implementation Details	3, 5

Habiba:

Report:	Slides:
System design, Explanation of AI usage	9, 11

Salma:

Report:	Slides:
Problem Description, Motivation	4, 8

We all worked on: Idea, Development Plan, and AI usage, more features (Slides 2, 6, 7, 10)

Milestone 2: All worked on it together via google meet, screen sharing, and our whatsapp group.

Slides: 14 - 20

Remaining Work and Next Steps

While the core system is fully functional, Milestone 3 will focus on significantly elevating the project from a working prototype to a polished, well-integrated academic tool. The following areas represent the remaining development work planned before the final submission.

1. Full System Integration and Workflow Refinement

Currently, each feature operates as a standalone module. The next step is to connect all functions into one seamless user workflow, where actions in one feature automatically trigger relevant updates in others. For example, completing a task via the Task Clock should automatically remove it from the Overload Detection calculation, trigger a check for newly unlocked Small Wins, and update the Procrastination Tracker, all without the user navigating through separate menu options. This end-to-end integration is essential for the system to feel like a cohesive product rather than a collection of separate tools.

2. Persistent Data Storage

One of the most significant limitations of the current version is that all task data is lost when the program closes. For Milestone 3, the team will implement file-based storage using C++ file I/O (`fstream`) to save all assignments to a `.txt` or `.csv` file and reload them automatically on the next launch. This will make the application genuinely useful across multiple study sessions rather than just within a single run.

3. Enhanced Priority Formula: Dynamic Recalibration

The current priority formula applies fixed weights to all tasks. For Milestone 3, we plan to introduce dynamic weight adjustment based on the user's academic profile. For example, if a student consistently has high-weight exams in one subject, the system will learn to slightly elevate urgency for that subject. Additionally, the formula will be updated to account for task clustering, if three assignments from different subjects are all due on the same day, each one's time pressure score will increase to reflect the compounded workload.

4. Subject-Level Organisation and Filtering

Currently all tasks appear in a single ranked list. The final version will allow the user to filter and view tasks by subject, giving a per-course workload breakdown. This will also feed into a new Subject Overload Warning that triggers when a single course is contributing disproportionately to the weekly stress index; a situation that often goes unnoticed until it is too late.

5. Burnout Auto-Reschedule Integration

The Burnout Estimator currently only reports a risk level. In Milestone 3, it will be connected directly to the scheduler: when a high burnout risk is detected, the system will automatically propose a revised weekly plan that spreads the workload more evenly, identifies which tasks are candidates for deferral based on their grade weight and deadline flexibility, and presents the user with a concrete recovery schedule rather than just a warning message.

6. Procrastination Tracker: Pattern-Based Suggestions

The current implementation counts delays and flags repeated avoidance. The Milestone 3 version will go further by analysing which types of tasks the user tends to delay, whether by subject, difficulty level, or estimated duration, and generating personalised advice based on those

patterns. For example, if the system detects that the user consistently delays tasks with a difficulty rating of 4 or 5, it will proactively suggest breaking those tasks into smaller subtasks upon entry and activate Small Wins Mode automatically.

7. Improved User Interface and Output Formatting

The current console interface is functional but basic. For the final submission, the team will improve the visual clarity of all output screens, add colour-coded text using ANSI escape codes to make urgency levels and warnings more immediately readable, and redesign the calendar view to display tasks grouped by day rather than a single ranked list. A summary dashboard will be added as the first screen shown on launch, giving the user an at-a-glance overview of today's tasks, this week's workload, and any active warnings.

8. Comprehensive Error Handling and Edge Case Testing

The team will conduct systematic testing across a wide range of edge cases, including tasks with identical priority scores, deadlines set in the past upon entry, zero-progress tasks with very long durations, and maximum-load scenarios with 20+ simultaneous assignments. Any bugs or unexpected behaviours identified during this phase will be fixed before the final submission, and the team will document the testing process and results in the final report.

9. Final Code Cleanup and Documentation

All code will be reviewed for consistency, unnecessary repetition will be refactored into helper functions, and every function will receive clear inline comments explaining its purpose, inputs, and outputs. The README on GitHub will be updated to reflect all final features and include step-by-step instructions for compiling and running on both Windows and Mac.